Plano de gravação e montagem do vídeo LexNexo

Guilherme Rossato Munhoz e João Henrique Souza Sabino | TDS-2026-038

Preparem oito clipes e montem um único vídeo de até 20 minutos. A ordem abaixo preserva os cinco blocos do professor: 3 minutos de contexto, 3 de arquitetura, 6 de demonstração, 5 de implementação e 3 de fechamento. A fala termina em 19:45; os 15 segundos restantes são margem, sem conteúdo novo.

Ordem e tempo final	Quem apresenta	Tela principal
01 — 00:00 a 03:00	Guilherme	Slides 1 e 2
02 — 03:00 a 04:00	Guilherme	Slide 3
03 — 04:00 a 06:00	João	Slides 3 e 4
04 — 06:00 a 12:00	Guilherme	Sistema no navegador
05 — 12:00 a 14:30	João	Editor e DER
06 — 14:30 a 17:00	João narra; Guilherme grava telas de apoio	Sistema, código e testes
07 — 17:00 a 17:40	Guilherme	Slide 5
08 — 17:40 a 19:45	João	Slide 6 e capa final

Trocas de tela na montagem

01 para 02: continuar nos slides e trocar para a arquitetura. 02 para 03: manter o mesmo diagrama e trocar a voz para João. 03 para 04: sair dos slides e entrar no navegador. 04 para 05: sair do sistema e abrir o editor. 05 para 06: intercalar as telas de apoio com a explicação técnica. 06 para 07: voltar ao slide de resultados. 07 para 08: trocar para limitações e encerrar na capa.

A gravação pode ser feita em outra ordem. Os números dos arquivos determinam a posição na edição, não obrigam vocês a gravar juntos ou em sequência.

Slides prontos e preparação da gravação

Slide	Conteúdo do slide	Quem usa
1	Capa: LexNexo, grupo 038, integrantes, curso, orientador e coorientador.	Guilherme na abertura; João no fim.
2	Problema, objetivo e público-alvo. Um fluxo simples: cliente > processo > prazo > documento > honorários.	Guilherme
3	Slide 3, Arquitetura do LexNexo: containers, tecnologias, implantação e comunicação frontend/API/banco.	Guilherme e João
4	Slide 4, Modelo de dados e módulos: recorte de Cliente, Processo, Contrato, Parcela, Agenda e Documento.	João
5	Resultados: fluxo demonstrado, 885 testes em 09/11 e cobertura de linhas 91,85% backend / 38,03% frontend.	Guilherme; João retoma os números nos testes.
6	Limitações, dificuldades, aprendizados e evolução. RF52 adiado e cobertura frontend abaixo da meta.	João

Abas prontas antes de apertar Gravar

Deixem abertos os slides, o LexNexo com sessão preparada, uma segunda sessão local de perfil restrito e o editor com os trechos marcados. No sistema, preparem cliente e processo fictícios novos, declaração com variáveis, PDF fictício, calendário de teste e contrato ainda não criado para esse processo. A gravação deve identificar se usa base local ou sistema publicado.

Gravação e corte

Usem a mesma resolução, preferencialmente 1920 x 1080, e fonte ampliada. Gravem dois segundos de respiro no início e no fim de cada arquivo e removam esses trechos na edição. Façam cortes diretos na troca de falante ou tela, sem acrescentar uma vinheta a cada clipe. A duração final inclui todas as transições, capa e créditos.

Copiem os dados fictícios de uma cola para reduzir digitação. Mantenham visível o resultado após salvar. Não mostrem senhas, tokens, .env, QR de autenticação ou dados de clientes reais. Preservem todos os registros de teste publicados. Anonimização e troca de credenciais ficam em base local separada.

Roteiro de João e ordem de gravação

Sua ordem de gravação: 03, 05, 06B e 08. Apresente backend, APIs e banco. Receba primeiro o arquivo 06A de Guilherme para narrar as telas no mesmo ritmo; deixe o 06B por último se essas capturas ainda não estiverem prontas.

Arquivo	Preparar e gravar	Destino na edição
03_joao_arquitetura_banco.mp4	Slides 3 e 4: API, PostgreSQL, módulos e relações.	04:00 a 06:00
05_joao_codigo_banco.mp4	Editor e DER: operação HTTP, validação, modelos, permissões e transação.	12:00 a 14:30
06B_joao_narracao_testes.mp4	Narração das telas 06A e, ao final, sua tela de código/testes por 45 segundos.	14:30 a 17:00
08_joao_fechamento.mp4	Slide 6 e capa: limites, dificuldades, aprendizados, evolução e agradecimento.	17:40 a 19:45

Trechos de código para deixar marcados

Arquivo	Trecho a destacar
frontend/src/services/api.js	Uma requisição relacionada ao contrato, como entrada do fluxo.
backend/advocacia/views/financeiro.py	ContratoViewSet, perform_create, transação e _gerar_parcelas.
backend/advocacia/serializers.py	ContratoSerializer e validação dos relacionamentos/valores.
backend/advocacia/models.py	Processo, Contrato, Parcela e seus vínculos.
backend/advocacia/mixins.py, permissoes.py e sigilo.py	Escopo por escritório, perfil e restrição de processo sigiloso.
backend/advocacia/tests/test_financeiro.py	Teste de geração/centavos ou rollback. Resultado e CI já executados.

Clipe 03 Arquitetura, backend e modelo de dados

João | 04:00 a 06:00 no vídeo final | arquivo: 03_joao_arquitetura_banco.mp4

04:00 a 05:00

Na tela: Slide 3: C2 e implantação, percorrendo navegador, frontend, API e banco com o ponteiro.

Fala-base: A API utiliza Python, Django e Django REST Framework, e os dados são armazenados em PostgreSQL. O frontend envia requisições HTTP; a API valida a entrada, aplica permissões e acessa o banco pelo ORM. Django oferece uma estrutura para modelos, validações e autenticação, e PostgreSQL permite manter relacionamentos e transações. A implantação usa Vercel para o frontend, Render para a API e Supabase para banco e armazenamento quando configurado. No C2, essas unidades aparecem separadas. A versão e o ambiente desta gravação são os que identificamos na abertura.

05:00 a 06:00

Na tela: Slide 4: recorte dos relacionamentos de Cliente, Processo, Contrato, Parcela, Agenda e Documento. Explicar os módulos e o escopo por escritório.

Fala-base: Internamente, a API está organizada por domínios, como clientes, processos, agenda e financeiro. O banco mantém os vínculos entre essas entidades. Um processo pertence a um cliente e pode ter um contrato; esse contrato reúne as parcelas. Documentos e eventos da agenda também se relacionam ao processo. O esquema é compartilhado entre os escritórios, e a API aplica o escopo do escritório nas consultas e nos vínculos. Este slide mostra um recorte dos relacionamentos. Agora vamos acompanhar esse caminho pela interface.

Transição para a demonstração

Termine com: "Com essa estrutura em mente, o Guilherme vai demonstrar o caminho de um cliente e seu processo pela interface". Deixe o slide 4 visível por um instante e corte diretamente para o navegador do clipe 04.

Clipe 05 Como a operação é processada

João | 12:00 a 14:30 no vídeo final | arquivo: 05_joao_codigo_banco.mp4

12:00 a 12:45

Na tela: Editor: estrutura backend/advocacia/views, services/api.js e o trecho do ContratoSerializer. Usar a operação de contrato como exemplo. Mostrar a requisição sem cabeçalhos secretos, se já houver captura segura.

Fala-base: O frontend utiliza uma camada de serviços para enviar a requisição à API. No backend, a rota chega ao viewset do domínio. O serializer verifica os campos e os relacionamentos; depois a operação é persistida pelos modelos do Django. A resposta volta para a interface, que mostra o resultado ou uma mensagem de validação. No contrato, por exemplo, a validação impede que a quantidade de parcelas produza valores menores que um centavo.

12:45 a 13:30

Na tela: DER ampliado e backend/advocacia/models.py. Apontar as relações de Processo, Contrato e Parcela, sem abrir dados reais no banco.

Fala-base: No banco relacional, o processo se vincula ao cliente e ao escritório. Documentos e eventos da agenda se relacionam ao processo; o contrato se relaciona ao processo e reúne suas parcelas. As chaves estrangeiras mantêm esses vínculos e o ORM representa as relações no código. Os escritórios compartilham o esquema: a separação é aplicada pelos vínculos de escritório e pelo escopo das consultas na API. Para os valores financeiros, usamos campos decimais.

13:30 a 14:00

Na tela: mixins.py: EscritorioScopedMixin.get_queryset e _filtrar_por_escritorio. Depois permissoes.py/sigilo.py e um pequeno recorte de LoginView/segunda etapa.

Fala-base: A autenticação utiliza tokens, e as permissões são verificadas na API. O escritório do usuário filtra as consultas e os vínculos recebidos também são conferidos. Processos sigilosos têm uma restrição adicional. Esconder um botão na interface não substitui essas verificações no backend.

14:00 a 14:30

Na tela: financeiro.py: @transaction.atomic, perform_create e _gerar_parcelas. Deixar o teste de rollback ao lado.

Fala-base: Contrato e parcelas são criados em uma transação. Se ocorrer falha durante a geração, a operação não deve deixar um contrato incompleto. O cálculo usa valores decimais, e a última parcela recebe o ajuste de centavos. Há um teste que simula a falha na segunda parcela e verifica o rollback.

Como apresentar sem perder tempo no editor

Abra os arquivos já nas funções relevantes, com zoom. Siga uma operação concreta em vez de percorrer pastas sem explicar. Mostre o vínculo no DER ao falar dos modelos; retorne ao trecho de permissão e ao financeiro. Não leia caminhos completos em voz alta nem abra o banco real para procurar dados. A montagem deve manter a sua voz contínua durante essas trocas.

Clipe 06 Recursos administrativos, integrações e testes

João narra; Guilherme grava telas de apoio | 14:30 a 17:00 no vídeo final | arquivo: 06_bloco_tecnico_montado.mp4

14:30 a 15:10

Na tela: Clientes: exportar dados do cliente fictício local e abrir o arquivo. Em outro cliente local separado, mostrar a confirmação de anonimização e o resultado. Apontar exportar_dados/anonimizar no código.

Fala-base: A exportação reúne os dados do titular. A anonimização é uma ação diferente e remove dados pessoais conforme o fluxo implementado. Estamos usando outro cadastro fictício na base local, para preservar os registros principais da demonstração.

15:10 a 15:35

Na tela: Sessão local do painel mestre: visão de escritórios e administração da plataforma, sem alterar plano ou permissões reais.

Fala-base: O painel mestre é separado do painel de um escritório. Ele atende à administração da plataforma, enquanto a equipe do escritório trabalha apenas no contexto e nas permissões correspondentes.

15:35 a 16:00

Na tela: Se pronto no ensaio: Assistente IA, pergunta curta sobre o processo fictício e resposta contextual. Mostrar provedor OpenAI no quadro técnico, sem chave.

Fala-base: O assistente utiliza a API da OpenAI com contexto controlado da aplicação. O texto gerado é um apoio e exige revisão humana antes de ser usado ou enviado.

16:00 a 16:15

Na tela: Se preparado: mostrar a notificação push recebida no dispositivo de teste e a mensagem na caixa de e-mail de teste. Sem recibos reais, mostrar o sino interno e declarar o limite.

Fala-base: O sino interno reúne os alertas da aplicação. Push e e-mail são canais externos: só consideramos entrega comprovada quando a mensagem chega ao destino de teste.

16:15 a 17:00

Na tela: Tabela datada de métricas, resultado salvo dos testes e um teste de rollback ou isolamento. Mostrar o workflow e a divisão de pastas do repositório.

Fala-base: Na execução final de 9 de novembro, passaram 885 testes: 550 Django, 290 Jest e 45 Playwright. O CI tem registro próprio e não se soma à execução local. A cobertura de linhas é 91,85% no backend e 38,03% no frontend, abaixo da meta de 70%. Também conferimos fluxos publicados e mantivemos os dados fictícios. Isso aumenta a confiança nos cenários testados, mas não significa ausência de falhas.

Montagem desse bloco

Use a voz de 06B durante os 2:30. Nos primeiros 1:45, cubra sua tela com o vídeo 06A do Guilherme. Aos 16:15 do vídeo final, retire essa sobreposição e mostre sua tela de testes. Resultado salvo e CI são evidência de execução anterior datada; não simule que a suíte inteira rodou durante esses 45 segundos.

IA, e-mail e push dependem de configuração e disponibilidade. Sem resposta ou recebimento reais, ajuste a fala para reconhecer o limite. Não use gravação simulada como comprovação externa.

Clipe 08 Limitações, aprendizados e encerramento

João | 17:40 a 19:45 no vídeo final | arquivo: 08_joao_fechamento.mp4

17:40 a 18:30

Na tela: Slide 6: pronto, limitações e backlog. Identificar RF52 e cobertura frontend abaixo da meta.

Fala-base: Também há limites. A notificação automática ao cliente a cada atualização do processo permanece no backlog. O PIX demonstrado é estático ou simulado; não há confirmação bancária automática. A cobertura de frontend precisa crescer, e ainda precisamos de avaliação de usabilidade com profissionais. Integrações externas só são apresentadas como validadas quando conseguimos demonstrar uma resposta ou recebimento real.

18:30 a 19:15

Na tela: Continuar no slide 6 e comentar as dificuldades de isolamento por escritório e consistência das parcelas.

Fala-base: Entre as principais dificuldades estiveram a separação de dados entre escritórios, as diferenças de permissão e a consistência das operações financeiras. Trabalhamos com desenvolvimento incremental, testes e revisões para corrigir problemas encontrados. O projeto nos ajudou a relacionar interface, API, banco e documentação, e a perceber que uma funcionalidade precisa ser verificada além do caminho em que tudo dá certo.

19:15 a 19:45

Na tela: Slide 1, usado como tela final: LexNexo, grupo 038, nomes e agradecimento.

Fala-base: Como próximos passos, pretendemos ampliar os testes da interface, validar a operação dos serviços externos e realizar sessões com profissionais de advocacia. O LexNexo responde ao problema de organização ao reunir a rotina do escritório em um ambiente integrado. Agradecemos aos orientadores e à banca e ficamos à disposição para perguntas.

Encerrar dentro do limite

Use o slide 6 ao explicar pendências, dificuldades e próximos passos. Na última cena, de 19:15 a 19:45, volte ao slide 1 e agradeça em nome dos dois. Os 15 segundos até 20:00 são margem; não acrescentem conteúdo para preencher esse espaço. A fala sobre dificuldades e aprendizados deve refletir a experiência real de vocês.

Ordem de montagem e conferência final

Na linha do tempo do editor, a sequência é 01 > 02 > 03 > 04 > 05 > 06 > 07 > 08. Os clipes 06A e 06B formam um único bloco 06. Exportem um único vídeo final; os arquivos separados são material de gravação.

Faixa de vídeo	Faixa de áudio
01 e 02: slides de Guilherme	Voz de Guilherme
03: slides de João	Voz de João
04: sistema operado por Guilherme	Voz de Guilherme
05: editor e DER de João	Voz de João
06, primeiros 1:45: telas 06A de Guilherme	Somente voz de João, arquivo 06B
06, últimos 45 s: testes de João	Continuação da voz de João
07: slide de resultados	Voz de Guilherme
08: limitações e capa final	Voz de João

Assistir ao arquivo exportado antes de enviar

Confiram a duração total de no máximo 20 minutos, volume semelhante entre as vozes, ausência de áudio duplicado no bloco 06 e legibilidade do código. Verifiquem se a criação foi seguida de confirmação, se o fluxo usa o mesmo caso fictício e se as limitações foram ditas sem contradizer a demonstração. Os dois devem participar e os cinco blocos do professor devem estar presentes.

Referência e uso

Este plano organiza o roteiro detalhado do grupo revisto em 09/11/2026. As ações, falas e limites vêm de docs/tcc/ROTEIRO_VIDEO_PREBANCA_2026.md e da orientação do professor fornecida em PDF. A divisão por integrante foi definida pelo proprietário. Use a apresentação editável entregue com estes roteiros. A execução dos recursos deve ser conferida no ensaio.

Este plano conserva a sequência temporal do roteiro existente, com uma mudança operacional: a explicação inicial do frontend fica no slide 3, e o editor entra no bloco de implementação. Os detalhes por cena podem ser adaptados no ensaio mantendo os cinco blocos, a veracidade da demonstração e o teto de 20 minutos.